ArrayList Study Guide (Java)

Solutions to the question bank + similar practice programs. Every program was compiled and run; outputs shown are real.

Quick reference: ArrayList methods

Method	What it does	Example
`add(x)`	Add at the end	`list.add(10);`
`add(i, x)`	Insert at index i (shifts the rest right)	`list.add(2, 99);`
`get(i)`	Read the element at index i	`list.get(0)`
`set(i, x)`	Replace the element at index i	`list.set(0, 99);`
`remove(i)`	Remove by index (int argument)	`list.remove(1);`
`remove(obj)`	Remove first matching value	`list.remove("B");` `list.remove(Integer.valueOf(40));`
`size()`	Number of elements	`list.size()`
`isEmpty()`	true if size is 0	`list.isEmpty()`
`clear()`	Remove all elements	`list.clear();`
`contains(x)`	true if value exists (case-sensitive for strings)	`list.contains("Jessy")`
`indexOf(x)`	First index of value, -1 if absent	`list.indexOf(30)`
`addAll(list2)`	Append all of another list	`list.addAll(second);`
`removeIf(cond)`	Remove all that match a condition	`list.removeIf(x -> x % 2 == 0);`
`Collections.sort(l)`	Sort ascending (add reverseOrder() for descending)	`Collections.sort(list);`
`Collections.reverse(l)`	Reverse in place	`Collections.reverse(list);`
`Collections.max/min(l)`	Largest / smallest element	`Collections.max(list)`
`Collections.frequency(l, x)`	Count occurrences of x	`Collections.frequency(list, 2)`

Exam tips

• Always add `import java.util.ArrayList;` (and `Scanner` / `Collections` when you use them). The class must be named `Main`.

• Indexes start at 0 and the last valid index is `size() - 1`. Going outside gives IndexOutOfBoundsException.

• `remove(1)` on an ArrayList<Integer> removes by **index**; use `remove(Integer.valueOf(x))` to remove by **value**.

• String comparisons in contains() / indexOf() are **case-sensitive** ("jessy" is not "Jessy").

• `System.out.println(list)` already prints in `[a, b, c]` format. For 'a b c' output, loop with a StringBuilder.

• Match the output text exactly: spaces, capital letters, full stops (for example `Element found.` vs `Found`).

• Do not remove inside a for loop over the same list; use `removeIf` or loop backwards.

• Read input with Scanner: `nextInt()` for numbers, `next()` for one word, `nextLine()` for a whole line.

Part 1: Question bank solutions (Q1 to Q10)

Q1. Create an ArrayList of integers, add 10, 20, 30 and print the list.

add() adds at the end; println(list) prints in [a, b, c] format.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        System.out.println(list);
    }
}
```

OUTPUT

```
[10, 20, 30]
```

Q2. Create an ArrayList of strings and print the first element.

get(0) returns the first element (index starts at 0).

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Asha");
        list.add("Rahul");
        list.add("Priya");
        System.out.println(list.get(0));
    }
}
```

OUTPUT

```
Asha
```

Q3. Add 3 numbers to an ArrayList and remove the second element.

remove(int index) removes by position. The second element is at index 1.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(5);
        list.add(15);
        list.add(25);
        list.remove(1);          // removes the element at index 1 (15)
        System.out.println(list);
    }
}
```

OUTPUT

```
[5, 25]
```

Q4. Replace the element at index 0 with the value 99.

set(index, value) replaces an existing element. add() would insert instead.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.set(0, 99);
        System.out.println(list);
    }
}
```

OUTPUT

```
[99, 20, 30]
```

Q5. Check if an ArrayList is empty and print "Empty" or "Not Empty".

isEmpty() returns true when size() is 0.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);              // remove this line to test the "Empty" case
        if (list.isEmpty()) {
            System.out.println("Empty");
        } else {
            System.out.println("Not Empty");
        }
    }
}
```

OUTPUT

```
Not Empty
```

Q6. Create an ArrayList of 5 names and print all names using a for-each loop.

for (String x : list) visits every element in order.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Asha");
        names.add("Rahul");
        names.add("Vicky");
        names.add("Jessy");
        names.add("Priya");
        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

OUTPUT

```
Asha
Rahul
Vicky
Jessy
Priya
```

Q7. Find the size of an ArrayList of fruits.

size() gives the number of elements (an ArrayList has size(), not length).

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Banana");
        fruits.add("Orange");
        System.out.println("Size: " + fruits.size());
    }
}
```

OUTPUT

```
Size: 4
```

Q8. Add "A", "B", "C", remove "B" and print the final list.

remove(Object) removes by value (here a String, so Java picks the value version).

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("A");
        list.add("B");
        list.add("C");
        list.remove("B");
        System.out.println(list);
    }
}
```

OUTPUT

```
[A, C]
```

Q9. Create an ArrayList, remove all elements, then check if it is empty.

clear() removes everything; then isEmpty() is true.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.clear();
        System.out.println(list.isEmpty());   // true
        System.out.println(list);             // []
    }
}
```

OUTPUT

```
true
[]
```

Q10. Add numbers 1 to 5 into an ArrayList and print the value at index 3.

Use a loop to add 1..5. Index 3 holds the 4th element, which is 4.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 1; i <= 5; i++) {
            list.add(i);
        }
        System.out.println(list.get(3));
    }
}
```

OUTPUT

```
4
```

Part 2: Similar practice programs (exam style)

These use input like the real tests do (N followed by N values) and are variations of the same ArrayList methods.

P1. Sum and average of the elements

Input: N, then N integers. Print the sum and the average (2 decimals). Cast to double before dividing.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        long sum = 0;
        for (int x : list) {
            sum += x;
        }
        double avg = (double) sum / n;
        System.out.println("Sum: " + sum);
        System.out.printf("Average: %.2f%n", avg);
    }
}
```

SAMPLE INPUT

```
5
10 20 30 40 50
```

OUTPUT

```
Sum: 150
Average: 30.00
```

P2. Largest and smallest element

Input: N, then N integers. Collections.max() and Collections.min() do the work.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        System.out.println("Max: " + Collections.max(list));
        System.out.println("Min: " + Collections.min(list));
    }
}
```

SAMPLE INPUT

```
6
12 -5 40 7 40 0
```

OUTPUT

```
Max: 40
Min: -5
```

P3. Reverse an ArrayList

Collections.reverse() reverses in place.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        Collections.reverse(list);
        System.out.println(list);
    }
}
```

SAMPLE INPUT

```
5
1 2 3 4 5
```

OUTPUT

```
[5, 4, 3, 2, 1]
```

P4. Sort ascending and descending

Collections.sort(list) is ascending. Add Collections.reverseOrder() for descending.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        Collections.sort(list);
        System.out.println("Ascending: " + list);
        Collections.sort(list, Collections.reverseOrder());
        System.out.println("Descending: " + list);
    }
}
```

SAMPLE INPUT

```
6
40 10 30 20 50 5
```

OUTPUT

```
Ascending: [5, 10, 20, 30, 40, 50]
Descending: [50, 40, 30, 20, 10, 5]
```

P5. Remove duplicates (keep first occurrence)

Build a new list and add a value only if contains() says it is not there yet.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        ArrayList<Integer> result = new ArrayList<>();
        for (int x : list) {
            if (!result.contains(x)) {
                result.add(x);
            }
        }
        System.out.println(result);
    }
}
```

SAMPLE INPUT

```
8
1 2 2 3 4 3 5 1
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

P6. Search an element and print its index

indexOf() returns the first index or -1 when the element is absent.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        int key = sc.nextInt();
        int index = list.indexOf(key);
        if (index != -1) {
            System.out.println("Found at index " + index);
        } else {
            System.out.println("Not found");
        }
    }
}
```

SAMPLE INPUT

```
5
10 20 30 40 50
30
```

OUTPUT

```
Found at index 2
```

P7. Insert a value at a given index

add(index, value) shifts the later elements right. Valid indexes are 0 to size().

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        int index = sc.nextInt();
        int value = sc.nextInt();
        if (index >= 0 && index <= list.size()) {
            list.add(index, value);
        }
        System.out.println(list);
    }
}
```

SAMPLE INPUT

```
4
10 20 30 40
2 99
```

OUTPUT

```
[10, 20, 99, 30, 40]
```

P8. Count how many times a value occurs

Collections.frequency(list, value) counts occurrences.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        int key = sc.nextInt();
        System.out.println(Collections.frequency(list, key));
    }
}
```

SAMPLE INPUT

```
7
2 5 2 8 2 5 1
2
```

OUTPUT

```
3
```

P9. Remove all even numbers

removeIf() takes a condition. Avoid calling remove() inside a normal for loop: indexes shift and elements get skipped.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        list.removeIf(x -> x % 2 == 0);
        System.out.println(list);
    }
}
```

SAMPLE INPUT

```
6
1 2 3 4 5 6
```

OUTPUT

```
[1, 3, 5]
```

P10. Second largest distinct element

Sort a copy, then walk backwards to the first value that differs from the maximum.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        ArrayList<Integer> copy = new ArrayList<>(list);
        Collections.sort(copy);
        int max = copy.get(copy.size() - 1);
        int second = max;
        for (int i = copy.size() - 2; i >= 0; i--) {
            if (copy.get(i) != max) {
                second = copy.get(i);
                break;
            }
        }
        if (second == max) {
            System.out.println("No second largest");
        } else {
            System.out.println(second);
        }
    }
}
```

SAMPLE INPUT

```
5
10 50 30 50 20
```

OUTPUT

```
30
```

P11. Merge two ArrayLists

addAll() appends every element of the second list to the first.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        int m = sc.nextInt();
        ArrayList<Integer> second = new ArrayList<>();
        for (int i = 0; i < m; i++) {
            second.add(sc.nextInt());
        }
        list.addAll(second);
        System.out.println(list);
    }
}
```

SAMPLE INPUT

```
3
1 2 3
2
10 20
```

OUTPUT

```
[1, 2, 3, 10, 20]
```

P12. remove(index) vs remove(value) on an Integer list

On ArrayList<Integer>, remove(1) removes the element at INDEX 1. To remove the VALUE 40 use remove(Integer.valueOf(40)). This is a very common trap.

CODE

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);
        list.remove(1);                     // by index -> removes 20
        System.out.println(list);           // [10, 30, 40]
        list.remove(Integer.valueOf(40));   // by value -> removes 40
        System.out.println(list);           // [10, 30]
    }
}
```

OUTPUT

```
[10, 30, 40]
[10, 30]
```

P13. Sort a list of names alphabetically

Collections.sort() on strings is alphabetical and case-sensitive (uppercase letters come before lowercase).

CODE

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<String> names = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            names.add(sc.next());
        }
        Collections.sort(names);
        System.out.println(names);
    }
}
```

SAMPLE INPUT

```
4
Rahul Aman Priya Karan
```

OUTPUT

```
[Aman, Karan, Priya, Rahul]
```

P14. Square every element in place, print space-separated

Use set(i, value) to update in place and a StringBuilder to print as 'a b c' (the format used in the mock paper).

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        for (int i = 0; i < list.size(); i++) {
            int v = list.get(i);
            list.set(i, v * v);
        }
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < list.size(); i++) {
            if (i > 0) sb.append(' ');
            sb.append(list.get(i));
        }
        System.out.println(sb);
    }
}
```

SAMPLE INPUT

```
5
1 2 3 4 5
```

OUTPUT

```
1 4 9 16 25
```

P15. Count even and odd numbers

Same as Mock Assessment 2 Q2. x % 2 == 0 is even (0 and negative evens too); everything else is odd.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        int even = 0, odd = 0;
        for (int x : list) {
            if (x % 2 == 0) even++;
            else odd++;
        }
        System.out.println("Even:" + even);
        System.out.println("Odd:" + odd);
    }
}
```

SAMPLE INPUT

```
5
1 2 3 4 6
```

OUTPUT

```
Even:3
Odd:2
```

P16. Search a name, remove the next one (mock-style)

Same idea as Mock Assessment 2 Q1: indexOf, then remove(index + 1) only if the match is not last.

CODE

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
        ArrayList<String> list = new ArrayList<>();
        list.add("Aman");
        list.add("Rahul");
        list.add("Sneha");
        list.add("Priya");
        list.add("Karan");
        int index = list.indexOf(name);
        if (index != -1) {
            System.out.println("Found");
            if (index < list.size() - 1) {
                list.remove(index + 1);
            }
        } else {
            System.out.println("Not Found");
        }
        System.out.println(list);
    }
}
```

SAMPLE INPUT

```
Rahul
```

OUTPUT

```
Found
[Aman, Rahul, Priya, Karan]
```